

Android Fragments API



In this article, we will look at the new Android UI API, the Fragments API. Part of the Android 3.0 SDK, it allows developers to use the large screen space available on tablets, efficiently, by dynamically updating a portion of the screen area without moving to a different screen space. Let's build a simple application to explore how Fragments API works.

Initially, the Fragments API was available only in the Android 3.0 SDK. To overcome criticism of 'Android not being backward-compatible', Google released a compatibility package that allowed previous versions of the API to support this new feature. Along with the Fragments API, this compatibility package also provides support for some other Google-specific APIs like market billing and market licensing.

Fragments, as the name suggests, are nothing but small chunks of the UI, which constitute a single *Activity*. They can be treated as individual *portlets* of a portal page. Though very similar to an *Activity* in terms of looks, lifecycle, etc., a fragment is different from an *Activity* in the sense that it should always reside in an *Activity*; fragments cannot exist independently.

Advantages of fragments

- It can modularise a single *Activity* into smaller units of the UI, making it easier to manage.
- A fragment need not be used with the same host *Activity*, but can be reused by other activities; and is hence very flexible.
- A fragment could exist without a UI screen, which means it could be used as an invisible worker for the *Activity*.
- A fragment allows us to maintain a 'back-stack' of its state, or transactions, so we can traverse its transactions when we press the *Back* button on the device.
- Makes the application compact, using less activities, by increasing the number of fragments in a single *Activity*.

The basic design principle we will use to build the sample application is very simple, and is illustrated in Figure 1. Before getting started on building the application, let's get a head start on the Fragments API. To create a *Fragment*, we need to extend the class with *Fragment*. There are different kinds of fragments available, such as a *ListFragment* (similar to a *ListActivity*), a *DialogFragment* (similar to a *Dialog Interface*) and a *PreferenceFragment* (similar to a *PreferenceActivity*).

Fragment callbacks

Let's discuss callbacks during the *fragment's* lifecycle (see Figure 2). As we already know, a *Fragment* cannot exist on its own; it has to be associated with an *Activity*. Therefore, the

Activity class needs to be extended with *FragmentManager*.

The initialisation of the fragment is done in the *onAttach()* method, which is invoked when the fragment is associated with an *Activity*. The *onCreate()* method is invoked when the fragment is created. A UI is associated with a fragment by invoking the method *onCreateView()*, which associates the *fragment id* with a UI layout. If you don't want to associate a UI with a fragment, then you can pass 'null'. Fragments are destroyed when the *Activity* is destroyed, and that's when the *onDestroyView()* and *onDetach()* methods are called.

Building the application

Now, let's get started with building our application, *EFY Group*. Figure 3 gives you the complete package structure of the application. It is clear that we will have two classes, one for the *Activity* (*FragmentManagerTestActivity*), and one for the fragment (*TestFragment*). In the beginning, I had discussed backward-compatibility support for fragments, in relation to which I have referenced an external jar file, `<android sdk home>/extras/android/support/v4/android-support-v4.jar`. This library provides the necessary API support for *Fragments*. In this application, I will be using the Android 2.2 library (Froyo) to demonstrate the Fragments API. I have uploaded the application to GitHub (<http://goo.gl/0WboW>), so you can get the code for the application there.

To begin, let's create an Android Project with the following specifications:

- Project Name: *FragmentManagerTest*
- Android Target: 2.2
- Application Name: *EFY Group*
- Package Name: *com.android.saket.fragments*
- CreateActivity: *FragmentManagerTestActivity*
- MinSDKVersion: 8 (OPTIONAL)

Then let's create another class, named *TestFragment*, with the superclass *android.support.v4.app.Fragment*.

Next, we set the layout for our application. Referring back to Figure 1, you can see that there is a *ListView* to list all the magazines published by the *EFY Group*, and an *ImageView* to display an image of the selected magazine. Figure 4 shows you the layout structure in *main.xml*. We can also create a folder, *layout-land*, and emulate the